Travel Order Resolver

Detection automatique du depart et de l'arrivee
dans des phrases en langage naturel

MS2 - PAR-15 T-IA901
Guermazi Majid
Mars 2026

1 Introduction

Ce rapport presente le travail realise dans le cadre du projet Travel Order Resolver. L'objectif est de construire une application qui recoit une phrase en langage naturel, identifie la ville de depart et la ville d'arrivee, puis calcule un itineraire ferroviaire en utilisant les horaires SNCF.

Le projet est decompose en deux parties. La premiere est un module NLP qui extrait les entites de type DEP (depart) et ARR (arrivee) depuis une phrase libre. La seconde est un moteur de routage base sur les donnees GTFS de la SNCF, qui calcule le meilleur trajet entre deux communes.

Ce rapport se concentre sur la partie NLP. Deux approches ont ete explorees et comparees : un pipeline base sur SpaCy, et un fine-tuning du modele pre-entraine CamemBERT.

2 Probleme et difficultes

La detection du depart et de l'arrivee dans une phrase presente plusieurs difficultes concretes.

Premierement, les noms de villes peuvent coincider avec des prenoms ou des noms communs. La phrase suivante illustre ce cas :

"Avec mes amis Florence et Paris, je voudrais aller de Paris a Florence."

Ici, Florence et Paris sont a la fois des prenoms et des villes. Un modele qui se base uniquement sur la reconnaissance de noms de villes commettrait des erreurs. Il doit comprendre le role semantique de chaque occurrence.

Deuxiemement, l'ordre des mots ne suffit pas. La ville de depart n'est pas toujours citee en premier, et les prepositions varient : "de X a Y", "depuis X vers Y", "X pour rejoindre Y", "je pars de X pour Y". Le modele doit apprendre la structure semantique, pas une position fixe.

Troisiemement, les utilisateurs n'ecrivent pas toujours correctement. Les majuscules, les traits d'union et les accents peuvent manquer. Une phrase comme "clermont-ferrand a lille" doit fonctionner de la meme facon que sa version correctement orthographiee.

3 Construction du dataset

3.1 Principe

Aucun dataset existant ne correspond a ce probleme specifique. Il a donc fallu en construire un. Le dataset est genere automatiquement a partir de templates de phrases et d'une liste de communes francaises issues du shapefile OpenStreetMap (36 000 communes).

Chaque phrase generee enregistre les positions exactes du depart et de l'arrivee dans la chaine de caracteres. Pour le fine-tuning de CamemBERT, les labels sont au format BIO : B-DEP, I-DEP, B-ARR, I-ARR, O.

3.2 Bug d'alignement des tokens

Un bug critique a ete identifie lors de la generation du dataset. Le tokenizer de CamemBERT (base sur SentencePiece) inclut l'espace precedant un mot dans l'offset du token.

Par exemple, pour :

"un train de La Framboisiere a Aiguilhe"

Le token representant 'La' a l'offset (11, 14), alors que la ville commence au caractere 12.

La correction consiste a parcourir tous les caracteres du span d'un token et a prendre le premier label non-O rencontre :

```
for char_pos in range(start, end):

    if char_labels[char_pos] != 'O':

        label = char_labels[char_pos]

        break
```

Apres correction, la distribution des labels dans le jeu d'entrainement est :

Label	Occurrences	Proportion
O	38 916	45.3 %
B-DEP	4 558	5.3 %
I-DEP	15 959	18.6 %
B-ARR	5 790	6.7 %
I-ARR	20 673	24.1 %

3.3 Exemples negatifs et cas ambigus

Pour forcer le modele a apprendre le contexte plutot que de simplement reconnaitre des noms de villes, plusieurs types d'exemples ont ete ajoutes :

- Phrases ou une ville est mentionnee mais ne constitue pas un depart ou une arrivee ("Mon ami habite a Lyon").
- Phrases avec distracteurs : un prenom identique a un nom de ville cite dans un contexte non-voyage.
- Phrases sans ville du tout ("Il fait beau aujourd'hui").

Ces exemples representent environ 10 % du jeu d'entrainement.

4 Approche 1 : SpaCy

4.1 Architecture

SpaCy est une bibliotheque NLP qui propose un pipeline modulaire : tokenizer, embeddings (tok2vec ou transformer), composant NER. Les labels DEP et ARR sont ajoutes manuellement. L'entrainement ne met a jour que le composant NER, les autres composants etant desactives.

Deux variantes ont ete testees : fr_core_news_sm (modele leger) et fr_core_news_md (modele plus performant).

4.2 Resultats et observations

Avec fr_core_news_sm et un dropout de 0.05, la loss est non decroissante : le modele ne converge pas.

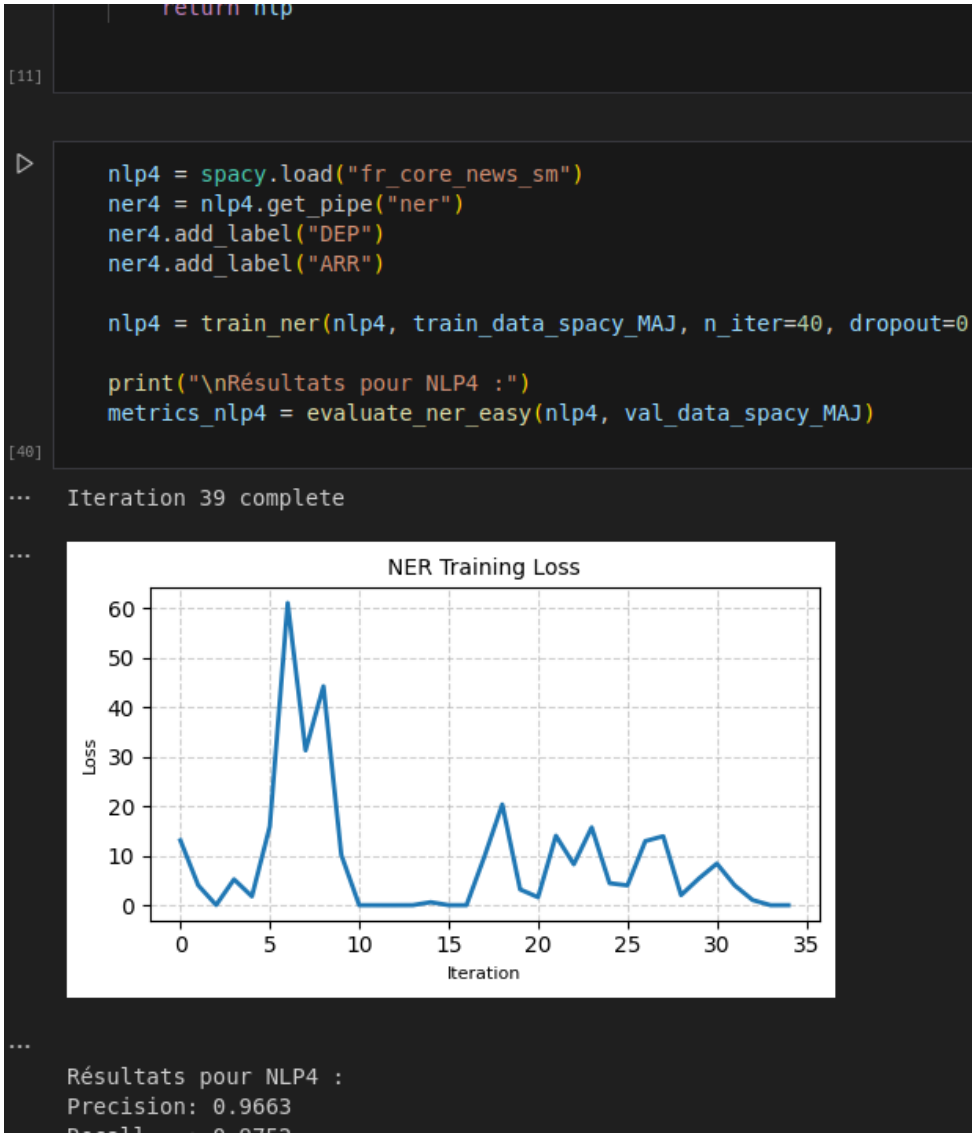

FIGURE 1 – Loss non decroissante avec fr_core_news_sm

Le passage a fr_core_news_md avec un dropout reduit a 0.03 resout ce probleme.

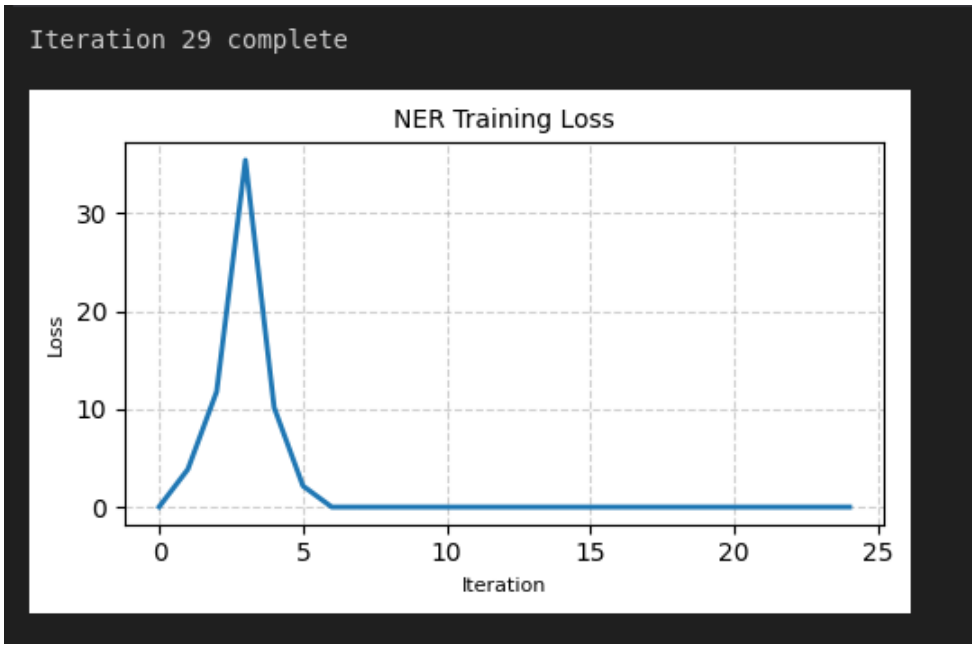

FIGURE 2 – Loss decroissante avec fr_core_news_md

Sur le jeu de validation, fr_core_news_md atteint un F1 de l'ordre de 0.97.

5 Approche 2 : CamemBERT fine-tune

5.1 Architecture

CamemBERT est un modele de type BERT pre-entraine sur un large corpus de texte francais.
Pour la tache NER, une tete de classification lineaire est ajoutee.
Elle produit un score pour chacun des 5 labels :
O, B-DEP, I-DEP, B-ARR, I-ARR.
Un dropout de 0.3 est applique pour limiter l'overfitting.

5.2 Format BIO

Le format BIO encode les entites multi-tokens.

```
je      -> O
veux    -> O
de      -> O
Lyon    -> B-DEP
a       -> O
Paris   -> B-ARR
```

5.3 Gestion du desequilibre de classes

Le label `O` domine le dataset (45 %). Sans correction, le modèle apprend à tout prédire `O`. Un WeightedTrainer applique des poids differents dans la fonction de perte :

```
CLASS_WEIGHTS = [0.1, 1.5, 1.0, 1.5, 1.0]
```

```
# O B-DEP I-DEP B-ARR I-ARR
```

5.4 Parametres d'entrainement

Parametre	Valeur
Modele de base	camembert-base
Learning rate	2e-5
Batch size	16
Epochs max	5
Weight decay	0.01
Warmup ratio	0.06
Dropout	0.3
Early stopping patience	3
Metrique	F1 (seqeval)

6 Resultats

6.1 Metriques globales

Precision	Recall	F1	Accuracy
0.9879	0.9956	0.9917	0.9988

6.2 Metriques par entite

Entite	Precision	Recall	F1
DEP	0.989	0.994	0.991
ARR	0.986	0.997	0.992

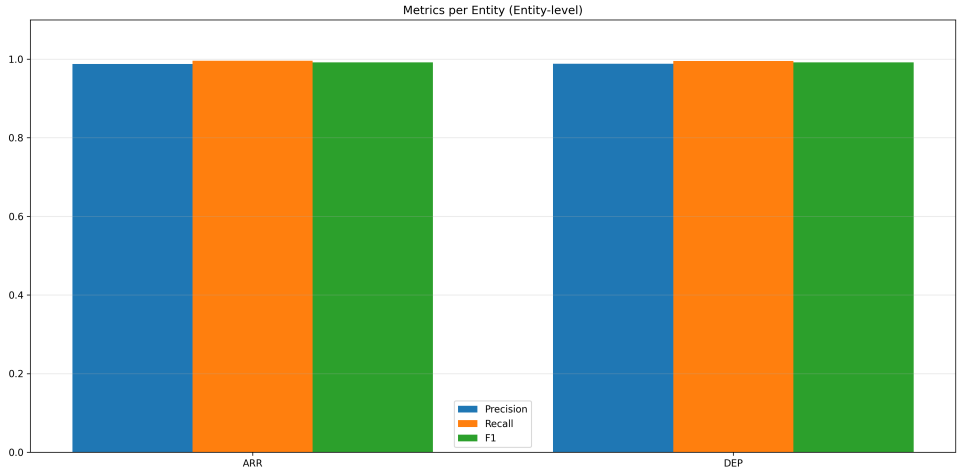

FIGURE 3 – Precision Recall F1 par entite

6.3 Matrice de confusion

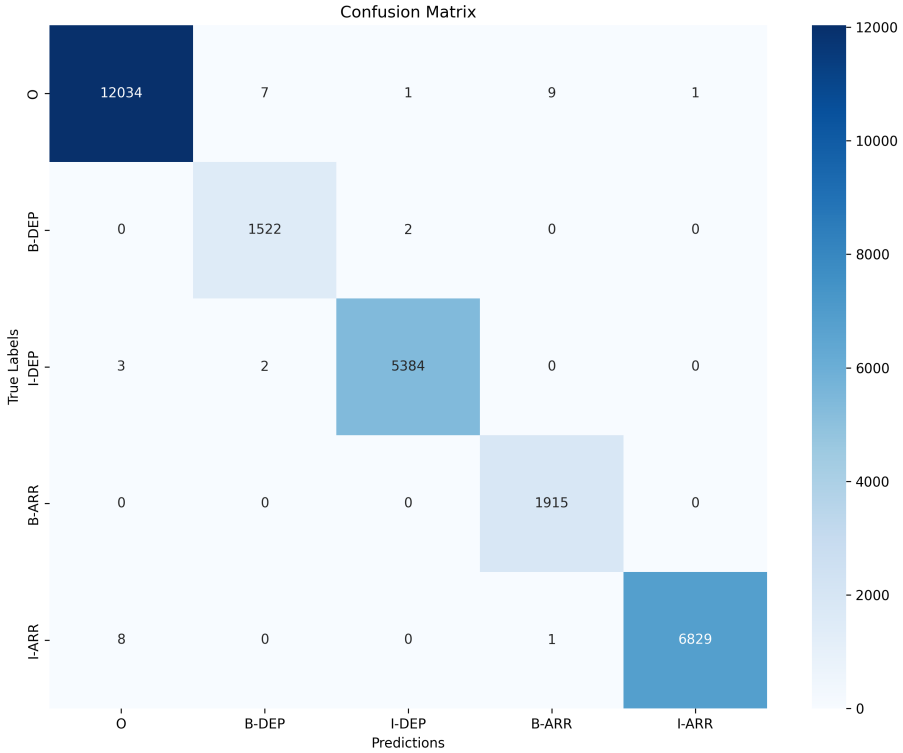

FIGURE 4 – Matrice de confusion token-level

7 Comparaison des approches

Modele	F1 DEP	F1 ARR	Convergence	Ambigus
SpaCy fr_core_news_sm	0.92	0.91	Non	Faible
SpaCy fr_core_news_md	0.96	0.95	Oui	Moyenne
CamemBERT	0.991	0.992	Oui	Elevee

CamemBERT obtient les meilleures performances sur tous les critères. SpaCy avec `fr_core_news_md` constitue une bonne alternative, plus simple à déployer et plus rapide à l'inférence, mais moins précise sur les cas ambigus.

8 Intégration avec le routage GTFS

L'application finale intègre le module NLP avec un moteur de routage ferroviaire. Le routage est basé sur l'algorithme CSA (Connection Scan Algorithm), adapté aux graphes de transport en commun avec horaires.

Le flux d'une requête est le suivant :

1. L'utilisateur saisit une phrase libre dans le terminal.
2. Le modèle NER extrait le départ (`DEP`) et l'arrivée (`ARR`).
3. Chaque ville est résolue en code INSEE via le shapefile des communes.
4. Les arrêts de train les plus proches de chaque commune sont identifiés (rayon de 50 km).
5. L'algorithme CSA calcule le trajet optimal (durée et correspondances minimales).

6. L'itinéraire est affiché avec horaires et noms de gares.

Exemple de résultat :
> je veux aller de Paris a Nice
Depart : Paris (89Arrivee : Nice (94Date : 2026-03-04
Depart Paris (commune) Arrivee Nice (commune) Depart choisi : 07 :00 – Arrivee : 12 :35 Duree : 335 min – Correspondances : 1

9 Limites et perspectives

Données synthétiques. Le dataset est entièrement synthétique. Bien qu'un effort ait été fourni sur la diversité des templates et des cas ambigus, il peut s'éloigner du langage réel utilisé par des voyageurs. Une collecte de phrases réelles améliorerait la robustesse.

Phrases longues. Les phrases de plus de 60 tokens montrent une légère baisse de performance. L'ajout de templates "bavards" avec plus de contexte dans le dataset permettrait d'adresser ce point.

Pistes d'amélioration. Parmi les perspectives identifiées : tester CamemBERT-large (gain estimé de 3 à 5 points de F1), augmenter le dataset à 15 000–20 000 phrases, gérer les arrêts intermédiaires (*"de Paris à Marseille en passant par Lyon"*), et ajouter une reconnaissance vocale en entrée.

10 Conclusion

Ce projet montre qu'il est possible de construire un systeme fiable d'extraction de villes dans des phrases libres, en construisant son propre dataset et en fine-tunant un modele pre-entraine.

La principale difficulte n'etait pas l'architecture du modele, mais la qualite du dataset.

CamemBERT fine-tune est robuste sur les cas ambigus et gere correctement les villes composees. Il constitue le meilleur compromis entre performance, généralisation et intégration pour ce projet.